

Day 10 : Image Handling And Assests

1. Introduction to Image Handling

Image Handling in Flutter is the process of displaying, loading, capturing, storing, and managing images within a Flutter application. Images can come from local assets, internet URLs, device storage, gallery, or camera. Flutter provides built-in widgets and packages to efficiently handle different types of images.

2. Why Do We use Image Handling?

- Makes apps visually attractive.
- Allows users to upload and view photos.
- Used in profile pictures, galleries, e-commerce apps, and social media apps.
- Improves user experience through visual content.

3. Image.asset()

Used to display images stored inside the application's assets folder.

- Works without an internet connection.
- Suitable for logos, icons, splash screens, and background images.
- Images must be declared in pubspec.yaml.

```
Image.asset("assets/images/photo.png")
```

4. Image.network()

Used to display images directly from a web URL.

- Requires an internet connection.
- Commonly used for profile pictures and product images from servers.
- Downloads the image every time unless caching is used.

```
Image.network(  
  "https://example.com/image.jpg"  
)
```

5. Image.file()

Displays images stored on the device.

- Mainly used with gallery and camera features.
- Reads images from local storage.

```
Image.file(File(imagePath))
```

6. BoxFit

Controls how an image fits inside its available space.

Additional Points

- Prevents image distortion.
- Helps maintain image quality.
- Different fit options provide different display styles.

```
Image.asset(  
  "assets/images/photo.png",  
  fit: BoxFit.cover,  
)
```

Common Types

- BoxFit.cover
- BoxFit.contain
- BoxFit.fill
- BoxFit.fitWidth
- BoxFit.fitHeight

7. Assets Folder Setup

Stores local images that are packaged with the application.

Folder Structure

```
assets/  
├── images/  
│   └── photo.png
```

Pubspec.yaml

```
flutter:  
  assets:  
    - assets/images/
```

8. image_picker Package

Allows users to select images from the gallery or capture images using the camera.

Additional Points

- Supports both Android and iOS.
- Returns the selected image path.
- Widely used for profile photo upload features.

```
dependencies:  
  image_picker: ^1.1.2
```

9. Gallery Picker

Allows users to choose an image from their phone gallery.

Additional Points

- Lets users use existing photos.
- Easy to implement.
- Commonly used in social media applications.

```
XFile? image =  
await picker.pickImage(  
  source: ImageSource.gallery,  
);
```

10. Camera Picker

Allows users to capture a new photo using the device camera.

Additional Points

- Opens the device camera directly.
- Useful for document verification and profile photos.
- Requires camera permission.

```
XFile? image =  
await picker.pickImage(  
  source: ImageSource.camera,  
);
```

11. Permissions

Provides access to device hardware and storage.

Additional Points

- Required before using camera features.
- Protects user privacy.
- Must be configured separately for Android and iOS.

```
<uses-permission  
  android:name="android.permission.CAMERA"/>
```

12. Display Picked Image

Shows the image selected by the user.

Additional Points

- Provides instant visual feedback.
- Useful before image upload.
- Helps users verify their selection.

```
Image.file(  
  File(image.path),  
)
```

13. cached_network_image

Loads and stores network images locally for faster access.

Additional Points

- Reduces internet usage.
- Improves app performance.
- Prevents repeated image downloads.

```
dependencies:  
  cached_network_image: ^3.4.1
```

14. Placeholder

Shows a temporary widget while the image is loading.

Additional Points

- Improves user experience.
- Indicates that content is loading.
- Prevents empty screen areas.

```
placeholder:  
(context, url) =>  
  CircularProgressIndicator()
```

15. Error Widget

Displays a widget if image loading fails.

Additional Points

- Handles invalid URLs.
- Prevents application crashes.
- Improves reliability.

```
errorWidget:  
(context, url, error) =>  
Icon(Icons.error)
```

16. Fade In Animation

Smoothly displays an image after it loads.

Additional Points

- Makes UI more attractive.
- Removes sudden image appearance.
- Creates a professional look.

```
fadeInDuration:  
Duration(seconds: 1)
```

17. Image Resizing

Adjusts image dimensions according to requirements.

Additional Points

- Saves screen space.
- Maintains proper layout.
- Improves UI consistency.

```
Image.asset(  
  "assets/photo.png"  
  width: 150,  
  height: 150,  
)
```

18. Image Cropping

Allows users to remove unwanted parts of an image.

Additional Points

- Useful for profile pictures.
- Improves image presentation.
- Helps focus on important content.

19. Image Filters

Applies visual effects to images.

Additional Points

- Enhances image appearance.
- Supports effects like grayscale and brightness.
- Useful in photo editing applications.

20. Profile Picture Upload

Allows users to upload their profile image.

Additional Points

- Improves personalization.
- Common feature in modern apps.
- Can use gallery or camera images.

21. Gallery Grid

Displays multiple images in a grid layout.

Additional Points

- Organizes images neatly.
- Easy image browsing.
- Commonly implemented using GridView

22. Image Viewer with Zoom

Allows users to zoom and inspect images.

Additional Points

- Improves image viewing experience.
- Useful for detailed images.
- Can be implemented using InteractiveViewer.

23. Image Preview Before Upload

Shows the selected image before sending it to the server.

Additional Points

- Reduces upload mistakes.
- Lets users confirm selection.
- Improves overall usability.

Code :

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

Run | Debug | Profile
void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: ImageDemo(),
      debugShowCheckedModeBanner: false,
    ); // MaterialApp
  }
}

class ImageDemo extends StatefulWidget {
  @override
  State<ImageDemo> createState() => _ImageDemoState();
}

class _ImageDemoState extends State<ImageDemo> {
  File? selectedImage;
  final ImagePicker picker = ImagePicker();

  Future<void> pickFromGallery() async {
    XFile? image =
      await picker.pickImage(source: ImageSource.gallery);

    if (image != null) {
      setState(() {
        selectedImage = File(image.path);
      });
    }
  }
}
```

```
Future<void> pickFromCamera() async {
  XFile? image =
    await picker.pickImage(source: ImageSource.camera);

  if (image != null) {
    setState(() {
      selectedImage = File(image.path);
    });
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Image Handling"),
    ), // AppBar
    body: SingleChildScrollView(
      child: Column(
        children: [
          SizedBox(height: 10),
          Text("Asset Image"),
          Image.asset(
            "assets/image/photo.jpg",
            height: 150,
          ), // Image.asset
          SizedBox(height: 20),
          Text("Network Image"),
          Image.network(
            "https://picsum.photos/200",
            height: 150,
          ), // Image.network
        ],
      ),
    ),
  );
}
```

```
    SizedBox(height: 20),

    Text("Network Image"),
    Image.network(
      "https://picsum.photos/200",
      height: 150,
    ), // Image.network

    SizedBox(height: 20),

    ElevatedButton(
      onPressed: pickFromGallery,
      child: Text("Pick From Gallery"),
    ), // ElevatedButton

    ElevatedButton(
      onPressed: pickFromCamera,
      child: Text("Take Photo"),
    ), // ElevatedButton

    SizedBox(height: 20),

    Text("Picked Image"),

    selectedImage != null
      ? Image.file(
          selectedImage!,
          height: 200,
        ) // Image.file
      : Text("No Image Selected"),
```